

VIEWS AND ITS USAGE

1. Create table and insert values

Firstly, Created the table Customers and Orders as we are going to do View operations.

```
1  use ECOM
2
3  -- Create Customers table
4  CREATE TABLE customers (
5      cust_id INT PRIMARY KEY,
6      first_name VARCHAR(50) NOT NULL,
7      last_name VARCHAR(50) NOT NULL,
8      email VARCHAR(100) UNIQUE,
9      phone_number VARCHAR(20),
10     address VARCHAR(255)
11 );
12
13 -- Create Orders table
14 CREATE TABLE orders (
15     order_id INT PRIMARY KEY,
16     cust_id INT,
17     order_date DATE NOT NULL,
18     total_amount DECIMAL(10, 2) NOT NULL,
19     status VARCHAR(50) DEFAULT 'Pending',
20     FOREIGN KEY (cust_id) REFERENCES customers(cust_id)
21 );
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

Messages

Commands completed successfully.

Completion time: 2025-08-14T20:20:50.5084679+05:30

```
SQLQuery2.sql ...U\Krishy (69)* SQLQuery1.sql... \Krishy (54)*
23 -- Insert data into the customers table
24 INSERT INTO customers (cust_id, first_name, last_name, email, phone_number, address) VALUES
25 (1, 'Alice', 'Johnson', 'alice.j@email.com', '555-1234', '123 Main St, Anytown'),
26 (2, 'Bob', 'Williams', 'bob.w@email.com', '555-5678', '456 Oak Ave, Somewhere'),
27 (3, 'Charlie', 'Brown', 'charlie.b@email.com', NULL, '789 Pine Rd, Otherplace');
28
29 -- Insert data into the orders table
30 INSERT INTO orders (order_id, cust_id, order_date, total_amount) VALUES
31 (101, 1, '2025-08-01', 75.50),
32 (102, 1, '2025-08-05', 150.00),
33 (103, 2, '2025-08-02', 25.75),
34 (104, 3, '2025-08-10', 200.25),
35 (105, 1, '2025-08-14', 50.00);
36
37 select * from customers
38 select * from orders
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

Results

	cust_id	first_name	last_name	email	phone_number	address
1	1	Alice	Johnson	alice.j@email.com	555-1234	123 Main St, Anytown
2	2	Bob	Williams	bob.w@email.com	555-5678	456 Oak Ave, Somewhere
3	3	Charlie	Brown	charlie.b@email.com	NULL	789 Pine Rd, Otherplace

	order_id	cust_id	order_date	total_amount	status
1	101	1	2025-08-01	75.50	Pending
2	102	1	2025-08-05	150.00	Pending
3	103	2	2025-08-02	25.75	Pending
4	104	3	2025-08-10	200.25	Pending
5	105	1	2025-08-14	50.00	Pending

Views

A view is a stored query that acts like a table. It's a way to encapsulate a complex SELECT statement and give it a name. When you query the view, the database executes the underlying SELECT statement and returns the result.

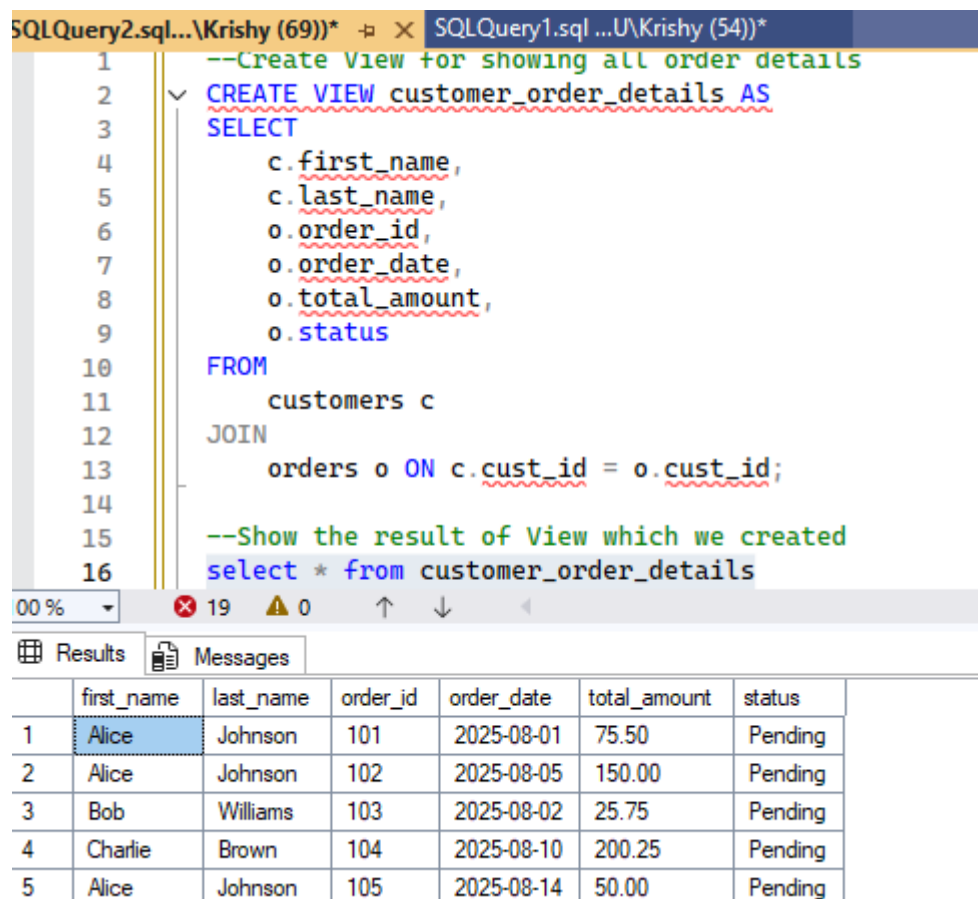
Now we will deep dive into views...

Usage

Firstly we create the view `customer_order_details` to show all orders placed using Select

Now, you can query this view as if it were a regular table:

```
SELECT * FROM customer_order_details;
```



The screenshot displays the SQL Server Enterprise Manager interface. The top pane shows the SQL script for creating and querying the view `customer_order_details`. The script is as follows:

```
--Create View for showing all order details
CREATE VIEW customer_order_details AS
SELECT
    c.first_name,
    c.last_name,
    o.order_id,
    o.order_date,
    o.total_amount,
    o.status
FROM
    customers c
JOIN
    orders o ON c.cust_id = o.cust_id;

--Show the result of View which we created
select * from customer_order_details
```

The bottom pane shows the results of the query, displaying a table with 7 columns: `first_name`, `last_name`, `order_id`, `order_date`, `total_amount`, and `status`. The table contains 5 rows of data:

	first_name	last_name	order_id	order_date	total_amount	status
1	Alice	Johnson	101	2025-08-01	75.50	Pending
2	Alice	Johnson	102	2025-08-05	150.00	Pending
3	Bob	Williams	103	2025-08-02	25.75	Pending
4	Charlie	Brown	104	2025-08-10	200.25	Pending
5	Alice	Johnson	105	2025-08-14	50.00	Pending

View with Complex SELECT

In order to view the total spending of the each customer, we created this view `customer_spending` with complex select

Now you can easily get this summarized data:

```
SELECT first_name, last_name, total_spent FROM customer_spending;
```

The screenshot shows the SQL Developer interface with two tabs: 'SQLQuery2.sql... \Krishy (69))' and 'SQLQuery1.sql ...U\Krishy (54))'. The active tab contains the following SQL code:

```
--Create view with complex select query to shows the total spending for each customer.
CREATE VIEW customer_spending AS
SELECT
  c.first_name,
  c.last_name,
  COUNT(o.order_id) AS number_of_orders,
  SUM(o.total_amount) AS total_spent
FROM
  customers c
JOIN
  orders o ON c.cust_id = o.cust_id
GROUP BY
  c.first_name, c.last_name;

--Show the result of View which we created
SELECT first_name, last_name, total_spent FROM customer_spending;
```

The 'Results' tab shows the output of the second query:

	first_name	last_name	total_spent
1	Charlie	Brown	200.25
2	Alice	Johnson	275.50
3	Bob	Williams	25.75

Views for Abstraction and Security

- Views can be used to restrict access to sensitive data. You can grant a user access to a view that shows only non-sensitive columns, while denying them direct access to the underlying table.
- Here we hide the customer address and email from the public users
- You can grant SELECT permission on public_customer_info to a user, but not on the customers table itself.

The screenshot shows the SQL Developer interface with a single tab containing the following SQL code:

```
--create a view that hides a customer's address and email for public users.
CREATE VIEW public_customer_info AS
SELECT
  cust_id,
  first_name,
  last_name,
  phone_number
FROM
  customers;

--Show the result of View which we created
select * from public_customer_info
```

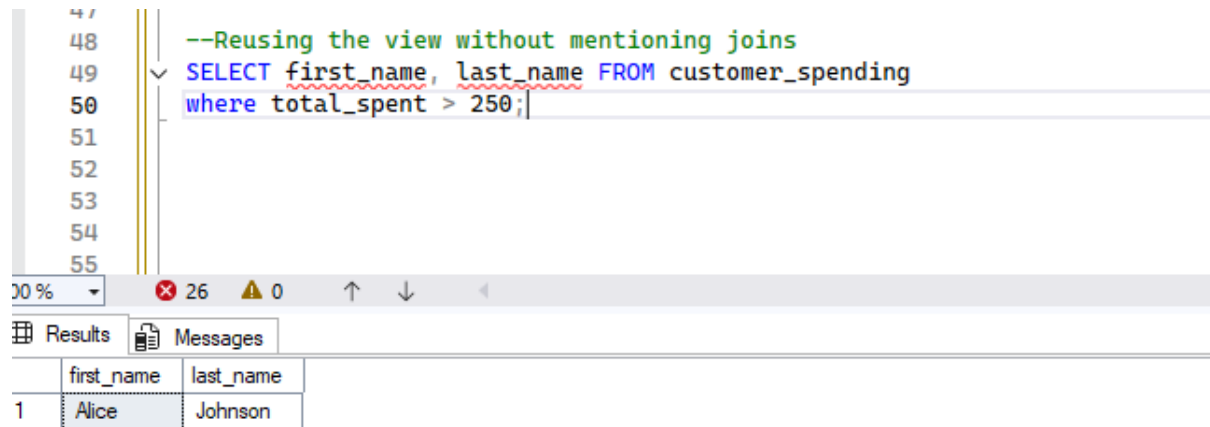
The 'Results' tab shows the output of the second query:

	cust_id	first_name	last_name	phone_number
1	1	Alice	Johnson	555-1234
2	2	Bob	Williams	555-5678
3	3	Charlie	Brown	NULL

Reusable SQL Logic

Instead of repeating a join in multiple queries, you can use the view itself .

Here I just directly used the view which I have created for Customer_spending earlier without using joins.



The screenshot shows a SQL IDE with a query editor and a results pane. The query editor contains the following SQL code:

```
--Reusing the view without mentioning joins
SELECT first_name, last_name FROM customer_spending
where total_spent > 250;
```

The results pane shows a table with two columns: first_name and last_name. The table contains one row with the values Alice and Johnson.

	first_name	last_name
1	Alice	Johnson

Updating Data through a View

In many cases, We can updating data through a view . However, there are certain strict limitations.

If a view is based on a single table and does not contain aggregations, GROUP BY, DISTINCT, or other complex clauses, you can often INSERT, UPDATE, and DELETE through it.

Example: Updatable View

This view is updatable because it's a simple selection from a single table.

51	
52	--creating view for Updating Data
53	CREATE VIEW updatable_customers AS
54	SELECT cust_id, first_name, last_name, email FROM customers;
55	
56	-- Updating data
57	UPDATE updatable_customers SET email = 'johnson.a@email.com' WHERE cust_id = 1;
58	
59	--Show the result of View which we created
60	select * from updatable_customers
61	
62	
63	
64	
65	
66	

00 %				
33 0				
Results Messages				
	cust_id	first_name	last_name	email
1	1	Alice	Johnson	johnson.a@email.com
2	2	Bob	Williams	bob.w@email.com
3	3	Charlie	Brown	charlie.b@email.com

Example: Non-Updatable View

This view is not updatable because it contains an aggregation and a GROUP BY clause.

Here we can see the error below...

60	select * from updatable_customers
61	
62	--creating view for non-Updating Data
63	CREATE VIEW non_updatable_view AS
64	SELECT cust_id, COUNT(order_id) AS total_orders FROM orders GROUP BY cust_id;
65	
66	
67	-- Updating data
68	UPDATE non_updatable_view SET cust_id = 0 WHERE total_orders = 3;
69	
70	
71	

100 %				
38 0				
Ln: 67 Ch: 1 SPC CI				
Messages				
Msg 4403, Level 16, State 1, Line 68				
Cannot update the view or function 'non_updatable_view' because it contains aggregates, or a DISTINCT or GROUP BY clause, or PIVOT or				
Completion time: 2025-08-14T21:08:58.9962174+05:30				

Drop a view

You can drop a view using drop view statement

```
69
70  --Drop a view
71  DROP VIEW customer_order_details;
72
73  select * from customer_order_details
74
75
```

100 % 39 0

Messages

Msg 208, Level 16, State 1, Line 72
Invalid object name 'customer_order_details'.

Completion time: 2025-08-14T21:29:56.1890919+05:30

With Check Option

The WITH CHECK OPTION clause prevents an UPDATE or INSERT statement from creating a row that the view cannot see. It ensures that all data modified through the view adheres to the view's WHERE clause.

Example

Let's create a view for orders with a total_amount greater than 100.

```
75  --create a view for orders with a total_amount greater than 100.
76  CREATE VIEW large_orders AS
77  SELECT order_id, cust_id, total_amount
78  FROM orders
79  WHERE total_amount > 100
80  WITH CHECK OPTION;
81
82  --Show the result of View which we created
83  select * from large_orders
84
85
86
87
88
89
```

% 44 0

Results Messages

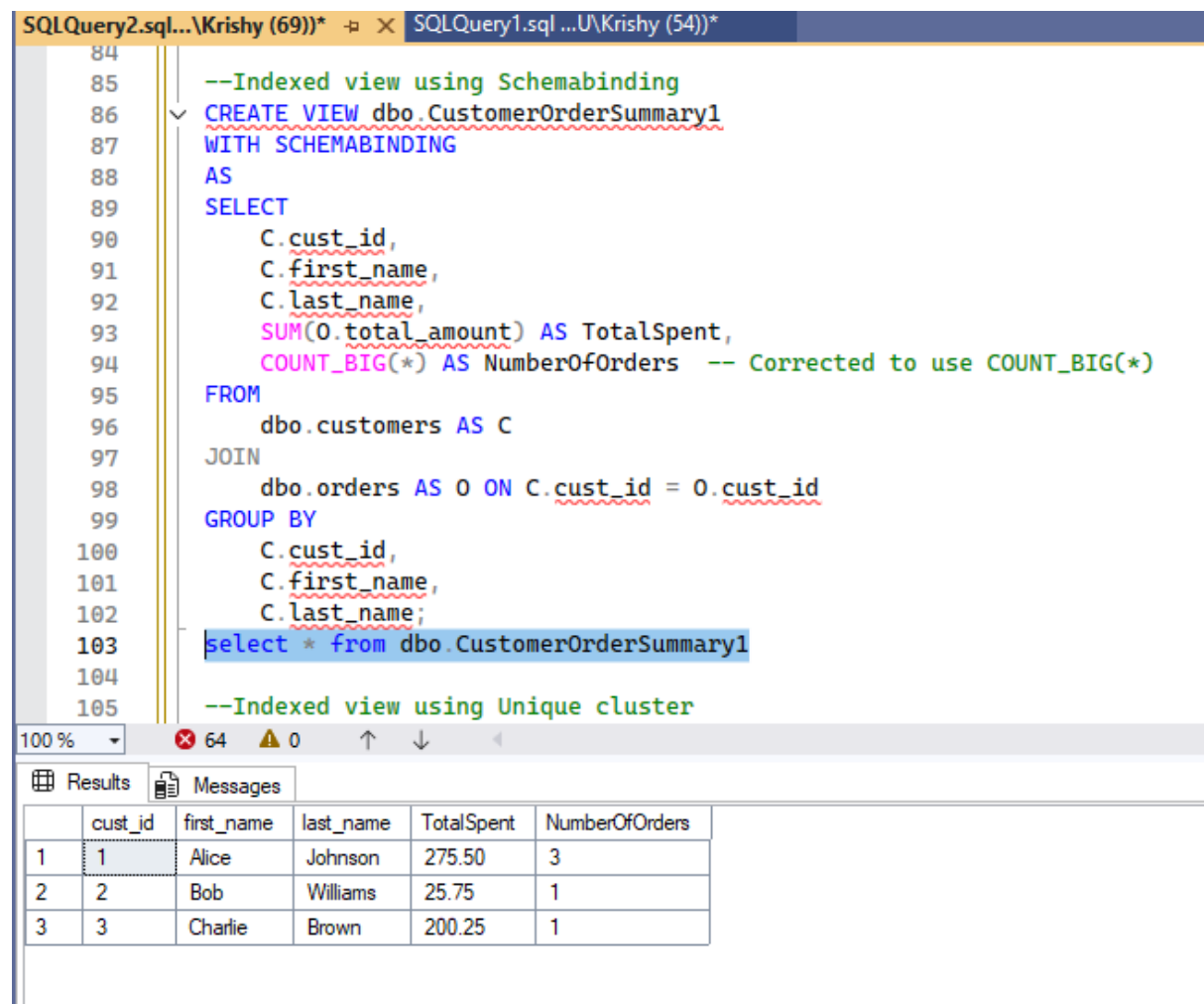
order_id	cust_id	total_amount
102	1	150.00
104	3	200.25

Creating an Indexed View

Let's create an indexed view that summarizes customer spending.

Create the view with SCHEMABINDING

We'll create a view that joins the customers and orders tables and aggregates the total spending. Note the use of COUNT_BIG(*) to count the number of orders, as required for indexed views.



```
84
85  --Indexed view using Schemabinding
86  CREATE VIEW dbo.CustomerOrderSummary1
87  WITH SCHEMABINDING
88  AS
89  SELECT
90      C.cust_id,
91      C.first_name,
92      C.last_name,
93      SUM(O.total_amount) AS TotalSpent,
94      COUNT_BIG(*) AS NumberOfOrders -- Corrected to use COUNT_BIG(*)
95  FROM
96      dbo.customers AS C
97  JOIN
98      dbo.orders AS O ON C.cust_id = O.cust_id
99  GROUP BY
100     C.cust_id,
101     C.first_name,
102     C.last_name;
103  select * from dbo.CustomerOrderSummary1
104
105  --Indexed view using Unique cluster
```

100 % 64 0

Results Messages

	cust_id	first_name	last_name	TotalSpent	NumberOfOrders
1	1	Alice	Johnson	275.50	3
2	2	Bob	Williams	25.75	1
3	3	Charlie	Brown	200.25	1

Create a UNIQUE CLUSTERED INDEX on the view

This step is what materializes the view. The database engine will now compute the results of the SELECT query and store them on disk. Any subsequent changes to the customers or orders tables will cause the indexed view's data to be automatically updated.

```
104
105 --Indexed view using Unique cluster
106 CREATE UNIQUE CLUSTERED INDEX IX_CustomerOrderSummary0_CustomerID
107 ON dbo.CustomerOrderSummary0 (cust_id);
108
109 select * from dbo.CustomerOrderSummary1
110
111
112
113 -- Create a materialized view
114 CREATE MATERIALIZED VIEW customer_spending_mv AS
115 SELECT
116     c.first_name,
117     c.last_name,
118     SUM(o.total_amount) AS total_spent
119 FROM
120     customers c
121 JOIN
```

% 65 0

Messages

Commands completed successfully.

Completion time: 2025-08-14T23:06:03.3189651+05:30

Materialized view

A materialized view is a physical table that stores the pre-computed result of a query. Unlike a regular view, a materialized view's data is stored on disk and can be refreshed manually or automatically at specific intervals.

Example: Materialized View (PostgreSQL syntax)

Database▼

Queries▼

Save & Share

Export▼

Run ▶

41 CREATE MATERIALIZED VIEW customer_spending_mv AS

42 SELECT

43 c.first_name,

44 c.last_name,

45 SUM(o.total_amount) AS total_spent

46 FROM

47 customers c

48 JOIN

49 orders o ON c.cust_id = o.cust_id

50 GROUP BY

51 c.first_name, c.last_name;

52

3 rows

first_name	last_name	total_spent
Bob	Williams	25.75
Alice	Johnson	275.50
Charlie	Brown	200.25

Thank you!